

Documentação do Projeto: Aplicativo de Controle de Tarefas (v1.0)

Integrante: Gabriel Moraes do Nascimento

Tema: 02-Controle de Tarefas

1. Introdução

Este projeto consiste no desenvolvimento de um aplicativo móvel utilizando o framework Flutter para gerenciamento de tarefas pessoais. O objetivo é permitir que o usuário gerencie suas atividades diárias com operações de CRUD (Criar, Ler, Atualizar e Excluir), contando com autenticação robusta, painel administrativo para gestão de usuários e visualização estatística.

2. Tecnologias Utilizadas

- **Linguagem:** Dart
- **Framework:** Flutter (v3.44.0)
- **Banco de Dados/API:** JSON Server (simulando API RESTful via `db.json`)
- **Gerenciamento de Estado:** `setState`
- **Interface:** Componentes customizados (modais de confirmação, formulários padronizados)

3. Estrutura do Projeto

O projeto foi organizado de forma modular para facilitar a manutenção e escalabilidade:

- **main.dart:** Ponto de entrada da aplicação.
- **screens/:** Contém as interfaces (`login_screen.dart`, `cadastro_screen.dart`, `home_screen.dart`, `admin_dashboard.dart`, `usertaskssreen.dart`, `config_screen.dart`).
- **api_service.dart:** Responsável pelas requisições HTTP e comunicação com a API.
- **user_data.dart:** Gerenciamento de sessão e persistência de dados do usuário logado.

4. Requisitos e Funcionalidades Implementadas

- **Autenticação e Segurança:** Sistema de login com validação de credenciais únicas e impedimento de cadastro de e-mails duplicados.
- **Privacidade e Individualidade:** Escopo de dados isolado por `usuarioId`, garantindo que cada usuário acesse apenas seus próprios registros.

- **Painel Administrativo:** Interface dedicada (`admin_dashboard.dart`) para monitoramento de usuários e tarefas, incluindo bloqueio de contas.
- **Exclusão em Cascata:** Ao remover um usuário, todas as suas tarefas vinculadas são removidas automaticamente do servidor, garantindo a integridade do banco de dados.
- **Interface de Confirmação:** Implementação de modais de segurança (`showDialog`) para todas as ações destrutivas (exclusão de tarefas e exclusão de conta), evitando perdas acidentais de dados.

5. Instruções para Execução

1. **Pré-requisitos:** Flutter SDK, Node.js e Git.
2. **Clonar:** `git clone https://github.com/Dev-Morais/controle_tarefas.git`
3. **Acessar:** `cd controle_tarefas`
4. **Instalar API:** `npm install -g json-server`
5. **Iniciar API:** `json-server --watch db.json`
6. **Executar:** `flutter run`

6. Conclusão

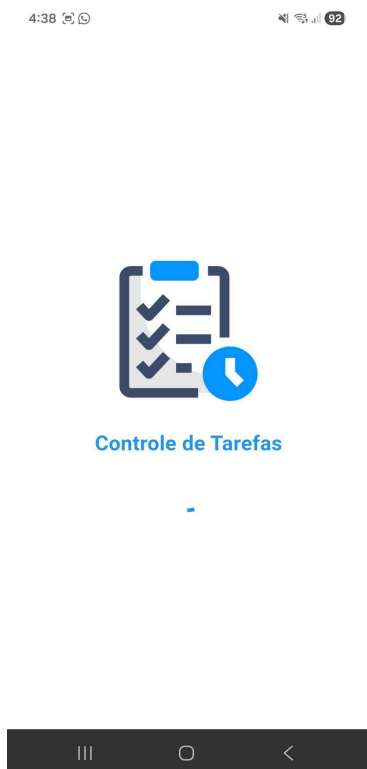
O desenvolvimento da aplicação permitiu a prática de conceitos avançados de consumo de APIs REST, manipulação de estados e construção de interfaces reativas no Flutter. A introdução de funcionalidades administrativas e a padronização das interações com o usuário elevaram o projeto a um nível profissional e seguro.

7. Anexo: Interface do Aplicativo

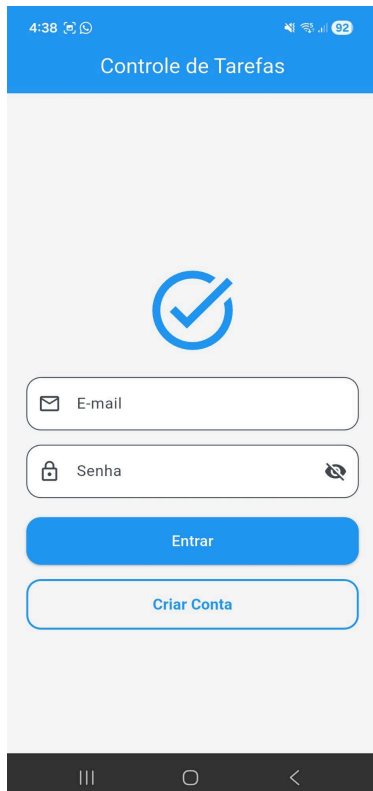
Esta seção ilustra os fluxos de trabalho do aplicativo.

7.1 Fluxo de Autenticação e Acesso

- **Figura 1:** Telas de inicialização



- **Figura 1.2:** Tela de login



- **Figura 2:** Tela de criação de nova conta

4:48 91

← Criar Conta

Nome

E-mail

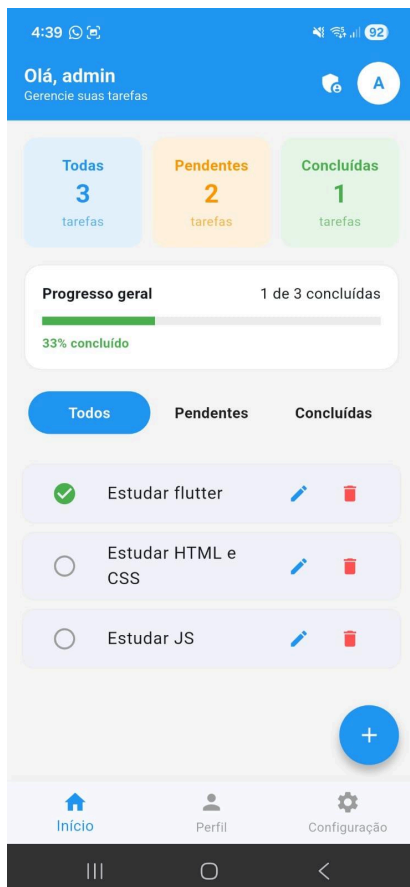
Senha

Cadastrar

III ○ <

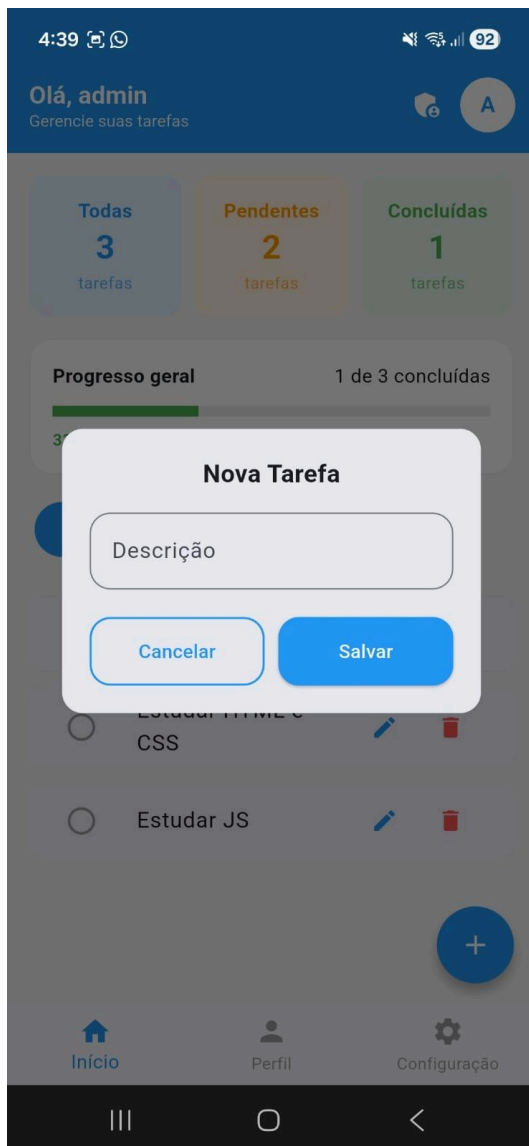
7.2 Dashboard e Produtividade

- **Figura 3:** Painel principal



7.3 Interação e Gestão de Tarefas (CRUD)

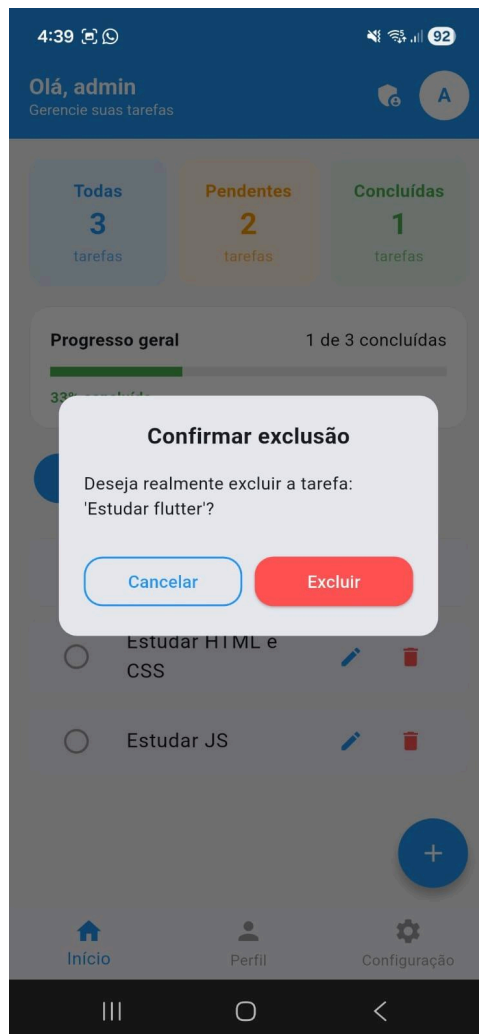
- **Figura 4:** Modais de nova tarefa



- **Figura 4.1:** Modais editar tarefa



- **Figura 4.2:** Modais excluir tarefa

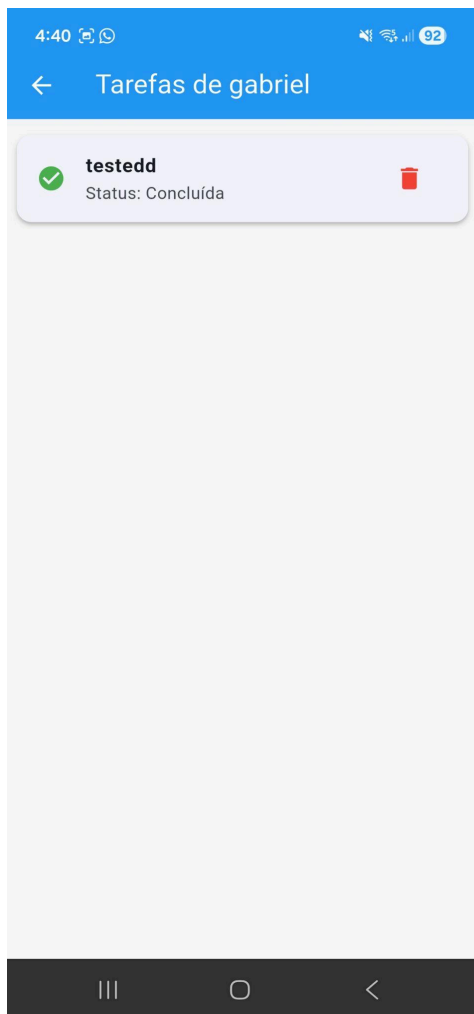


7.4 Painel do Administrador

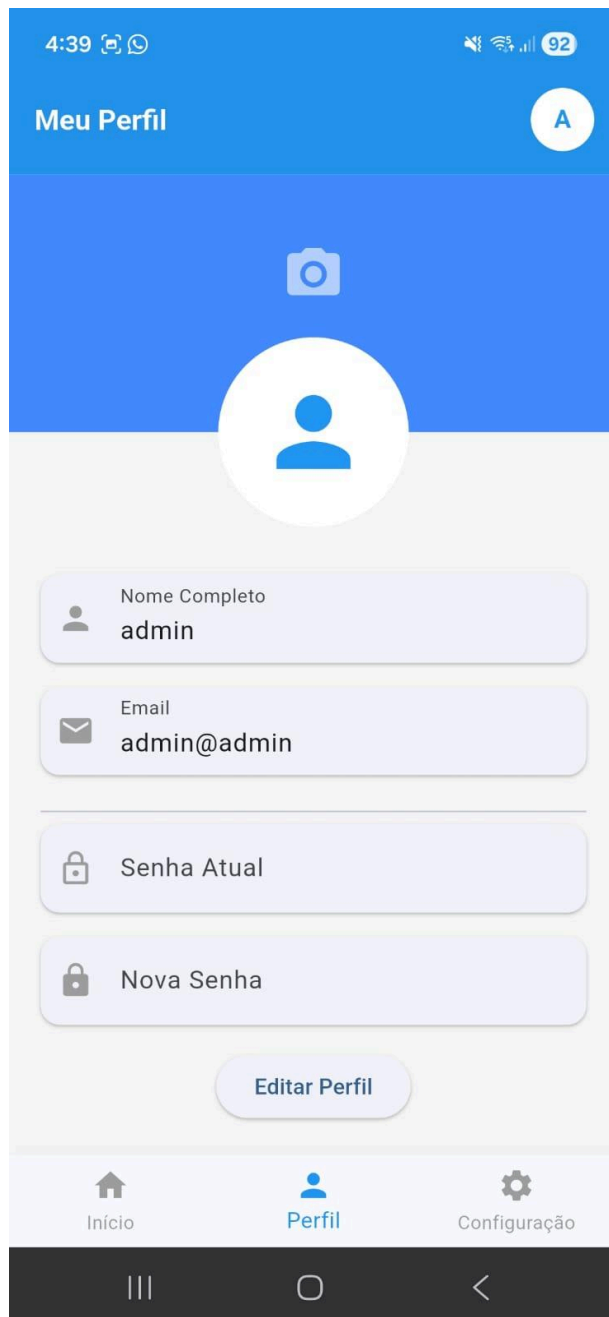
- **Figura 5:** Painel administrativo



- **Figura 5.1:** visualização de tarefas de usuários



7.5 Tela de perfil:



7.6 Tela de configuração:

